

DevOps. Automatización y monitorización de infraestructuras

Unidad 04.

Automatización remota con Salt y monitorización de sistemas



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 04. Automatización remota con Salt y monitorización de sistemas	2
1. 💡 Introducción	2
2. 🏗️ Orquestación de Infraestructura con Terraform	2
3. 🏠 Gestión de la configuración: El modelo SaltStack	3
4. 👁️ Fundamentos de la observabilidad	4
5. ⚙️ El ecosistema de monitorización bajo el capó	5
5.1 Prometheus (La base de datos temporal)	5
5.2 Grafana (La capa de visualización)	5
5.3 Los exporters (El sistema nervioso)	5
6. 🚨 Alertas proactivas y PromQL	6
7. 🔄 Integración de entornos (El puente Unidad 03 -> Unidad 04)	7
8. ★ Buenas prácticas en automatización y monitorización	8
9. 📖 Webgrafía	8

UNIDAD 04. AUTOMATIZACIÓN REMOTA CON SALT Y MONITORIZACIÓN DE SISTEMAS

1. INTRODUCCIÓN

En las unidades anteriores (UD02 y UD03), resolvimos el problema de cómo definir hardware virtual como código mediante Vagrant y cómo desplegar sistemas operativos a nivel de metal de forma masiva utilizando FOG Project y el arranque PXE. Sin embargo, en el mundo real, encender los servidores es solo el principio del ciclo de vida de la infraestructura.

Una vez que tienes decenas o cientos de servidores operativos en tu centro de datos, surgen dos problemas críticos de escalabilidad operativa que hacen inviable la administración manual (entrar por SSH máquina por máquina):

1. **La deriva de la configuración:** ¿Cómo garantizas que el servidor web 1 y el servidor web 50 tienen exactamente la misma versión de Nginx, el mismo archivo de configuración y las mismas contraseñas de administrador sin hacerlo a mano?
2. **La ceguera operativa:** ¿Cómo sabes si a las 3:00 AM un servidor se ha quedado sin memoria RAM, si un disco duro está al 99% de su capacidad o si la red está experimentando cuellos de botella?

Para dar respuesta a estos inmensos desafíos, esta unidad da un salto hacia la modernidad DevOps. Abandonaremos las tareas manuales y la virtualización pesada para adentrarnos en la orquestación ligera con **Terraform**, implementaremos un ecosistema de automatización gobernado por **SaltStack**, y desplegaremos un cerebro analítico compuesto por el tándem **Prometheus + Grafana** para dotar a nuestra red de observabilidad total en tiempo real.

2. ORQUESTACIÓN DE INFRAESTRUCTURA CON TERRAFORM

Hasta ahora hemos utilizado Vagrant como nuestro director de orquesta. Vagrant es excepcional para entornos de desarrollo local y para emular máquinas completas con BIOS y Kernel propios (VirtualBox), lo que nos acercaba más a simular máquinas reales.

Sin embargo, en esta unidad nuestra infraestructura da un giro hacia arquitecturas más ágiles basadas en contenedores Docker. Para gestionarlos, introducimos **Terraform**.

Terraform es una herramienta de Infraestructura como Código (IaC) de HashiCorp. Su mayor virtud es que es completamente agnóstica: mediante el uso de "Providers" (proveedores), puede interactuar con nubes públicas (AWS, Azure), hipervisores (VMware) o motores locales como Docker, todo utilizando el mismo lenguaje declarativo (HCL - HashiCorp Configuration Language).

En nuestro laboratorio, no levantamos contenedores tecleando ***docker run*** en la terminal. Todo se define en el archivo maestro ***terraform.tf***, donde declaramos el estado deseado de nuestra infraestructura:

- **Redes híbridas:** Terraform se encarga de crear una red interna en Docker

(*devops_lab_net_ud04*) y la enlaza directamente a la interfaz puente física **LabCEFIRE** (172.28.0.0/24) que reside en el Host. Esto permite que los contenedores tengan direcciones IP reales dentro del laboratorio y puedan hablar con las máquinas virtuales de unidades anteriores.

- **Volúmenes persistentes:** Los contenedores son efímeros; si se destruyen, los datos desaparecen. Terraform declara volúmenes físicos (*ud04_salt_master_data*, *ud04_prometheus_data*, etc.) y los ancla a los contenedores. Si el contenedor del servidor de monitorización cae, al levantarlo de nuevo, el historial de métricas seguirá intacto.
- **Inyección de configuración:** Terraform no solo levanta la máquina, sino que utiliza bloques upload para inyectar archivos de configuración vitales (como *prometheus.yml* o *alerts.yml*) directamente dentro del contenedor antes siquiera de que el proceso principal arranque.

3. GESTIÓN DE LA CONFIGURACIÓN: EL MODELO SALTSTACK

SaltStack (habitualmente conocido como Salt) es un motor de automatización, gestión de la configuración y ejecución remota diseñado para ser increíblemente rápido y escalable (capaz de manejar decenas de miles de servidores simultáneamente).

A diferencia de herramientas como Ansible (que es *agentless* y se conecta por SSH secuencialmente), Salt utiliza un modelo basado en eventos y una arquitectura **Master-Minion** que se comunica a través de un bus de mensajes de ultra-alta velocidad (ZeroMQ).

La arquitectura

- **El Salt Master:** Es el cerebro central y el servidor de control. Almacena el estado deseado de la infraestructura, los archivos de configuración y las políticas de seguridad. En nuestro laboratorio, el contenedor *vm-master* asume este rol crítico.
- **Los Salt Minions:** Son agentes ligeros instalados en cada nodo que queremos gobernar (nuestros contenedores *vm-minion-1* y *vm-minion-2*). A diferencia de otros sistemas, el Master no busca a los Minions; son los Minions los que, al arrancar, buscan la IP del Master en la red y se conectan a él, quedando a la escucha perpetua en modo Pull.

El Paradigma de confianza (PKI y Claves)

Salt no confía en nadie por defecto. Para que un Minion pueda recibir órdenes de un Master, debe existir un apretón de manos criptográfico férreo:

1. Cuando instalamos el paquete *salt-minion* en un nodo, este genera automáticamente un par de claves RSA (pública y privada).
2. El Minion envía su clave pública al Master solicitando acceso.
3. Esta clave queda en un estado Unaccepted (Pendiente). El servidor no interactuará con esa máquina hasta que un administrador humano, utilizando el comando *salt-key -A -y*

en el Master, acepte explícitamente la clave.

4. Una vez aceptada, toda la comunicación (comandos y retorno de datos) viaja cifrada mediante AES.

Ejecución Remota al Instante

El verdadero poder de Salt se revela cuando la topología está conectada. Si queremos saber qué máquinas están vivas en la red, no hacemos un ping manual a cada IP. Lanzamos una orden global desde el Master:

```
salt '*' test.ping
```

El comodín * indica que la orden va dirigida a todos los Minions registrados. El bus ZeroMQ envía la orden en milisegundos, y las respuestas vuelven a la terminal del Master de forma asíncrona, permitiendo auditar cientos de máquinas en un par de segundos.

4. FUNDAMENTOS DE LA OBSERVABILIDAD

La monitorización tradicional se limitaba a luces verdes y rojas: "¿El servidor responde al ping? Sí/No". La **observabilidad** moderna va mucho más allá: consiste en comprender los estados internos de un sistema complejo a partir de las salidas (métricas, logs y trazas) que este expone hacia el exterior.

Para lograr esto, la industria ha pivotado de modelos antiguos hacia arquitecturas más resilientes. El debate principal reside en cómo viajan los datos desde el servidor monitorizado hasta el servidor central:

- **El modelo Push (Empujar):** Es el enfoque antiguo. Tienes un agente en cada servidor que lee la CPU y "empuja" agresivamente esos datos al servidor central. Si tienes 1,000 servidores empujando datos al mismo tiempo, el servidor central sufre cuellos de botella masivos. Además, si el servidor central cae, los agentes locales empiezan a acumular datos en memoria hasta colapsar sus propias máquinas.
- **El modelo Pull (Tirar):** Es el paradigma moderno, abanderado por Prometheus. En este modelo, los servidores simplemente exponen una página web en texto plano (usualmente en el puerto 9100, ruta /metrics) con sus números actuales. Prometheus actúa como un rastreador web silencioso; lee su agenda de contactos (***prometheus.yml***), visita las URLs de los servidores cada 15 segundos, "tira" de los datos, los comprime y los guarda. Si Prometheus cae, los servidores no se enteran ni sufren ningún impacto de rendimiento.

5. EL ECOSISTEMA DE MONITORIZACIÓN BAJO EL CAPÓ

Nuestro laboratorio despliega un *stack* de observabilidad modular. En lugar de tener una aplicación monolítica pesada que intente hacer todo, usamos piezas altamente especializadas que se comunican entre sí.

5.1 Prometheus (La base de datos temporal)

Prometheus es el corazón del ecosistema. No tiene una interfaz gráfica deslumbrante; su verdadera magia reside en su motor interno: una **Base de Datos de Series Temporales (TSDB)**.

Una TSDB está diseñada para un propósito: ingerir millones de números asociados a una marca de tiempo (timestamp) de la forma más rápida y comprimida posible. Cuando Prometheus hace un scrape (raspado) de un servidor, recoge métricas crudas (ej. **`node_memory_MemFree_bytes 104857600`**) y les añade etiquetas multidimensionales (**`instance="vm-minion-1", job="node"`**). Esto permite filtrados y agrupaciones complejas a velocidades vertiginosas.

5.2 Grafana (La capa de visualización)

Dado que Prometheus almacena datos puros que son difíciles de interpretar a simple vista por un humano, necesitamos "ojos". **Grafana** es la plataforma líder mundial en visualización de datos de código abierto.

Grafana no guarda métricas, lee información de distintas fuentes (*Datasources*). Para mostrar datos de nuestro sistema, se conecta a Prometheus (configurándose como un *Datasource*) y le lanza consultas. En nuestro laboratorio, hemos automatizado la configuración de Grafana para que, al arrancar, cargue mediante su API REST un **Dashboard completo llamado "UD04 - System Overview"**. Este panel de control traduce los números de Prometheus en ricas gráficas dinámicas de líneas y medidores (gauges) que muestran el pulso de la CPU, la salud de la memoria y la saturación de los discos en tiempo real, etc.+

5.3 Los exporters (El sistema nervioso)

Prometheus, por sí solo, no sabe cómo leer la RAM de un servidor o el tráfico de una tarjeta de red. Necesita traductores. Estos servicios, colocados en las máquinas de destino, extraen métricas del sistema y las exponen en formato web (texto plano) para que Prometheus pase a recogerlas. En nuestra topología, desplegamos tres tipos:

1. **Node Exporter (Puerto 9100):** Es el agente estándar para máquinas Linux (sean físicas o virtuales). Lo instalamos mediante scripts en el Master y los Minions. Se adentra en el Kernel de Linux (leyendo las carpetas `/proc` y `/sys`) para exponer datos críticos: porcentajes de CPU inactiva, memoria Swap utilizada, inodos disponibles en disco, bytes transmitidos por las interfaces de red, etc.
2. **cAdvisor (Puerto 8080):** Desarrollado por Google, Container Advisor está especializado exclusivamente en Docker. A diferencia del Node Exporter, cAdvisor se monta directamente sobre el socket del demonio de Docker en el Host (`/var/run/docker.sock`).

Su misión es diseccionar contenedor por contenedor, indicando exactamente qué contenedor (ej. vm-minion-2) está consumiendo qué cantidad de recursos aislados.

3. **Blackbox Exporter (Puerto 9115):** Mientras que los anteriores miran "hacia dentro" (monitorización de caja blanca), Blackbox mira "hacia fuera". Simula el comportamiento de un usuario o de la red. Lo utilizamos para realizar sondeos activos (probes): lanza pings (ICMP) a los servidores para medir la latencia de respuesta, o hace peticiones HTTP contra puertos específicos para verificar que un servicio web está devolviendo un código 200 OK y no un error 500.

6. ALERTAS PROACTIVAS Y PROMQL

La observabilidad no consiste en tener a un administrador de sistemas mirando fijamente una pantalla gráfica las 24 horas del día. Un sistema bien diseñado debe ser lo suficientemente inteligente como para analizar sus propios datos en segundo plano y gritar cuando algo va mal, o mejor aún, cuando algo va a ir mal.

Para lograr esto, Prometheus no utiliza SQL clásico, sino que emplea su propio lenguaje funcional multidimensional: **PromQL (Prometheus Query Language)**. PromQL no busca filas y columnas; evalúa Vectores Instantáneos (el valor exacto en este milisegundo) y Vectores de Rango (una ventana de valores a lo largo del tiempo).

Mediante nuestro archivo de reglas (alerts.yml), le hemos inyectado inteligencia al sistema evaluando fórmulas complejas en tiempo real. A continuación, diseccionamos cómo PromQL lee la infraestructura bajo el capó:

- **Filtros Regex y detección de caídas (InstanceDown):**
 - **La consulta principal (expr):**
 - `up{job=~".*-node"} == 0`
 - **Análisis PromQL:** La métrica up no viene del servidor; es un indicador sintético que el propio Prometheus genera con valor 1 si el raspado (scrape) hacia un objetivo tuvo éxito y 0 si falló (es decir, el objetivo no responde). En lugar de escribir una regla estática para cada servidor individual, PromQL permite usar expresiones regulares (`=~".*-node"`). Esto significa que la alerta se aplicará dinámicamente a cualquier trabajo de monitorización (job) cuyo nombre termine en "-node", permitiendo que el sistema escale sin reescribir reglas.
- **Matemática vectorial (CriticalMemoryUsage):**
 - **La consulta principal (expr):**
 - `(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100 > 95`
 - **Análisis PromQL:** Los Exporters (como Node Exporter) envían métricas crudas y absolutas; no envían un "porcentaje de memoria usada" precalculado. PromQL permite realizar álgebra directamente entre diferentes métricas (vectores). El motor toma los bytes disponibles en este instante, los divide por los bytes totales reportados por el sistema, resta el resultado a 1 (para obtener la fracción de uso real) y lo multiplica por 100 para convertirlo en un porcentaje. Si el resultado de toda esa operación matemática supera el valor umbral de 95, la alerta se evalúa como verdadera.

- **Regresión lineal y proactividad (DiskWillFillIn24Hours):**
 - **La consulta principal (expr):**
 - `predict_linear(node_filesystem_free_bytes{mountpoint="/"}[6h], 24*3600) < 0`
 - **Análisis PromQL:** Este es el mayor exponente de la proactividad. Al añadir [6h] al final de la métrica de espacio libre en disco, convertimos un dato plano (un vector instantáneo) en un Vector de Rango (una lista de todos los valores reportados en las últimas 6 horas). La función nativa ***predict_linear*** toma todo ese historial, analiza la velocidad exacta a la que se ha ido llenando el disco y aplica un algoritmo de regresión lineal para proyectar la línea de tendencia 86.400 segundos (24*3600) hacia el futuro. Si el resultado matemático de esa proyección temporal es menor que 0 (lo que significaría disco lleno), dispara la alerta un día entero antes de que el desastre ocurra físicamente.
- **Análisis de contadores y eventos del Kernel (OOMKillerInvoked):**
 - **La consulta principal (expr):**
 - `increase(node_vmstat_oom_kill[5m]) > 0`
 - **Análisis PromQL:** En Prometheus, los Counters (contadores) son un tipo de métrica que, por diseño, solo puede crecer hacia arriba (o resetearse a cero si el servidor se reinicia). Si el kernel aniquila un proceso por falta de RAM (Out Of Memory Killer), el contador pasará de 0 a 1, y se quedará en 1 indefinidamente. Si usáramos la regla simple `node_vmstat_oom_kill > 0`, la alerta sonaría para siempre tras el primer evento. La función matemática `increase()[5m]` soluciona esto calculando exclusivamente el incremento (la diferencia) que ha sufrido el contador durante la ventana móvil de los últimos 5 minutos. Si detecta que el contador ha sumado nuevos eventos recientes, indica que el kernel está matando procesos activamente en este momento.

7. INTEGRACIÓN DE ENTORNOS (EL PUENTE UNIDAD 03 -> UNIDAD 04)

Uno de los mayores retos para un arquitecto de sistemas moderno (Ingeniero DevOps) es lidiar con entornos heterogéneos. Nuestro alumnado rara vez administra un centro de datos puro; lo normal es gestionar una mezcla de ordenadores físicos antiguos, máquinas virtuales tradicionales y clústeres de contenedores. En este laboratorio, no construimos silos aislados.

De forma optativa en el laboratorio, hemos diseñado un orquestador de puente (***03-integrate-ud03-to-ud04.sh***) cuya misión es unificar arquitecturas.

El script localiza las máquinas virtuales pesadas (Legacy) gobernadas por Vagrant en la Unidad 03 (nuestro servidor FOG y la Golden Image) y realiza un asalto controlado:

1. Se inyecta por SSH en ellas.
2. Instala el agente ***salt-minion*** y lo configura para apuntar a la IP de nuestro contenedor maestro de Terraform (172.28.0.40).
3. Instala el ***prometheus-node-exporter*** para abrir las métricas del sistema.

4. Realiza el registro y la aceptación de las claves de forma automática en el Master.

El resultado: Las máquinas virtuales aisladas se convierten en nodos de pleno derecho dentro de nuestro panel de Grafana. Podemos enviarles comandos de automatización desde el mismo cerebro que controla los contenedores efímeros, logrando un control absoluto de un entorno híbrido sobre la red **LabCEFIRE**.

8. ★ BUENAS PRÁCTICAS EN AUTOMATIZACIÓN Y MONITORIZACIÓN

Para que infraestructuras de esta envergadura no se vuelvan frágiles, los scripts de despliegue y las configuraciones de alertas deben obedecer a principios arquitectónicos estrictos:

1. **Idempotencia:** Todos los scripts de provisión de Salt, Node Exporter, Prometheus y Grafana de este laboratorio están diseñados para ser idempotentes. Antes de instalar un paquete, comprueban si el binario ya existe (**command -v**). Antes de arrancar un demonio, matan proactivamente procesos zombie previos (**pkill -9**). Esto permite que el administrador ejecute el script diez veces seguidas de forma segura, garantizando que el sistema acabará siempre en el estado deseado sin corromperse ni lanzar errores de "puertos en uso".
2. **Mitigación de la "Fatiga de alertas" (Alert fatigue):** Es el enemigo mortal de la monitorización. Si el sistema está configurado para enviar un correo electrónico cada vez que la CPU llega al 90% durante un segundo, el administrador recibirá miles de alertas "basura". Eventualmente, su cerebro ignorará los avisos, y cuando un disco duro realmente muera, nadie lo verá. En nuestro archivo **alerts.yml**, utilizamos la directiva **for: 5m**. Esto obliga a que la condición anómala (ej. CPU > 80%) se mantenga sostenida durante 5 minutos continuos antes de molestar al humano, filtrando así los picos temporales inofensivos.
3. **Desacoplamiento de datos (Persistencia):** Como se vio en la fase de Terraform, la computación debe ser efímera, pero la historia debe persistir. Nunca confíes el almacenamiento de bases de datos temporales (Prometheus) o la configuración de cuadros de mando (Grafana) al sistema de archivos interno de un contenedor Docker. Un simple reinicio los purgaría. Mapear volúmenes lógicos externos garantiza la resiliencia de la observabilidad.
4. **Silencio no interactivo:** Cuando gestionas flotas de servidores, no puede haber pantallas azules esperando a que alguien pulse "Intro". El uso de variables como **export DEBIAN_FRONTEND=noninteractive** y los flags de confirmación automática (**apt-get install -y -qq**) son obligatorios para que la cadena de montaje no se paralice.

9. 📖 WEBGRAFÍA

- **Salt Project Official Documentation** 🔗 <https://docs.saltproject.io/>
 - Documentación oficial del ecosistema SaltStack. Fundamental para profundizar en la arquitectura del bus ZeroMQ, el modelo Master/Minion, la gestión avanzada de claves criptográficas y la redacción de Módulos de Estado (.sls).
- **Prometheus** 🔗 <https://prometheus.io/docs/introduction/overview/>

- Guía principal sobre la filosofía de monitorización basada en extracción (Pull Model). Contiene referencias esenciales para dominar el lenguaje de consultas PromQL y la configuración avanzada de la Time-Series Database (TSDB).
- **Grafana Dashboards**  <https://grafana.com/docs/grafana/latest/>
 - Documentación oficial sobre cómo conectar fuentes de datos (Datasources) a Grafana, construir paneles dinámicos y gestionar la observabilidad visual mediante provisionamiento automatizado (JSON y APIs).
- **Terraform by HashiCorp**  <https://developer.hashicorp.com/terraform>
 - Guía de referencia para escribir Infraestructura como Código (IaC) de forma declarativa utilizando el lenguaje HCL, con especial enfoque en los providers para gestionar redes, volúmenes y contenedores en entornos Docker locales.